SecureShop Security Report

1. Executive Summary

This security report documents the DevSecOps implementation and initial security assessment of the SecureShop microservices platform. SecureShop is a lightweight e-commerce application consisting of six microservices orchestrated with Docker Compose behind an Nginx API gateway.

Project Scope:
- System: SecureShop microservices application
- Assessment Date: May 4, 2026
- Security Tools: Bandit (SAST), pip-audit (SCA), manual secrets scanning, planned container/DAST scanning

Key Findings:
- 7 medium-severity issues identified through SAST analysis
- All issues are configuration-related (binding interfaces, missing timeouts)
- No hardcoded secrets detected
- No known vulnerabilities in declared Python dependencies
- All findings are remediation-ready

2. Methodology

Tools & Practices

Practice	Tool(s)	Status
SAST	Bandit for Python	Executed
SCA	pip-audit for Python	Executed
Secrets Scanning	Pattern-based manual scan	Executed
Container Scanning	Trivy (planned)	Pending docker-compose up
DAST	OWASP ZAP baseline (planned)	Pending docker-compose up
IaC Scanning	Checkov (planned)	Pending execution

Scope

- Python services: User, Order, Notification, Inventory
- Node.js services: Product, Payment (npm audit pending)
- API gateway: Nginx configuration
- Orchestration: docker-compose.yml

3. Findings Summary

SAST Findings (Bandit)

Total: 7 Medium-Severity Issues

Configuration Issues (B104) - 4 instances
- **Files:** services/inventory/app.py, services/notification/app.py, services/order/app.py, services/user/app.py
- **Issue:** Flask applications bound to 0.0.0.0 (all interfaces)
- **Risk:** Unintended network exposure in production
- **Remediation:**
  - Use environment-based host binding: `host=os.getenv('FLASK_HOST', '127.0.0.1')`
  - In production, bind to specific interfaces or use reverse proxy
  - Add documentation requiring 127.0.0.1 binding for development

HTTP Timeout Issues (B113) - 3 instances
- **Files:** services/order/app.py (lines 21, 25, 29)
- **Issue:** requests library calls without timeout parameters
- **Risk:** Potential for hanging connections and DoS vulnerability
- **Remediation:** Add timeout parameter to all requests: `requests.get(..., timeout=10)`

SCA Findings

Python Dependencies: No known vulnerabilities
- Flask 2.3+: Current version
- PyJWT 2.8+: Current version
- requests 2.31+: Current version

Node.js Dependencies: npm audit pending

4. Threat Model Correlation

STRIDE Threats vs. Findings

STRIDE Category	Threat	Finding	Mitigation
Spoofing	Invalid JWT tokens, weak auth	None found	JWT signature validation enabled
Tampering	Unauthorized service calls	Requests without timeout	Add timeout parameters
Repudiation	Missing audit trail	No SAST findings	Implement structured logging
Information Disclosure	Exposed interfaces, secrets in code	0.0.0.0 binding, no secrets	Restrict network exposure
Denial of Service	Hanging requests, no rate limits	Missing timeouts	Add timeouts and rate limiting
Elevation of Privilege	Unauthorized access	No auth bypass found	Maintain JWT validation

5. Recommendations

Immediate Actions (High Priority)

1. **Fix Network Binding**
   ```python
   host = os.getenv('FLASK_HOST', '127.0.0.1')  # Default to localhost
   app.run(host=host, port=port)
   ```

2. **Add Request Timeouts**
   ```python
   response = requests.get(url, timeout=10)
   response = requests.post(url, json=data, timeout=10)
   ```

3. **Add Rate Limiting**
   - Implement rate limiting at API gateway level (Nginx)
   - Add per-service rate limiting (Flask-Limiter)

Medium-Term (Deployment)

1. **Secrets Management**
   - Use environment variables (already configured)
   - Consider HashiCorp Vault for production
   - Rotate JWT_SECRET regularly

2. **Container Hardening**
   - Use specific base image versions
   - Scan images with Trivy before deployment
   - Implement security scanning in CI/CD pipeline

3. **Logging & Monitoring**
   - Add request/response logging
   - Implement centralized log aggregation
   - Set up security event alerts

Long-Term (Operations)

1. **Continuous Security**
   - Run SAST/SCA on every commit
   - Conduct quarterly security reviews
   - Keep dependencies updated

2. **API Security**
   - Implement API versioning
   - Add request validation schemas
   - Enforce HTTPS in production

3. **Access Control**
   - Implement role-based access control (RBAC)
   - Add audit logging for sensitive operations
   - Regular access review cycles

6. Prioritization

Critical (Must Fix)

- None identified

High

- Add request timeouts to Order Service
- Restrict network binding in all services

Medium

- Add rate limiting configuration
- Implement centralized logging

Low

- Enhanced monitoring and alerting
- Performance optimization

7. Compliance & Standards

- OWASP Top 10 coverage: Input validation, secrets management
- CWE-605: HTTP Client Initialization with Hard-Coded IP Address
- CWE-1104: Use of Unmaintained Third Party Components

8. Next Steps

1. Generate SAST findings (Bandit) - **DONE**
2. Build and scan Docker images (Trivy)
3. Run DAST baseline scan (OWASP ZAP)
4. Execute IaC scanning (Checkov)
5. Update findings and generate final report
6. Plan remediation sprint